# Coding Exam 2 Preparation

# Pointers

# Function Descriptions

### 1. `swap_ints`

**Description**: Swap the values of two integers using pointers.

**Prototype**:

```
void swap_ints(int *a, int *b);
```

**Example**:

```
int x = 5, y = 10;
swap_ints(&x, &y); // Now x = 10, y = 5
```

---

### 2. `swap_chars`

**Description**: Swap the values of two characters using pointers.

**Prototype**:

```
void swap_chars(char *a, char *b);
```

**Example**:

```
char a = 'A', b = 'B';
swap_chars(&a, &b); // Now a = 'B', b = 'A'
```

---

### 3. `find_max`

**Description**: Find the maximum value in an integer array using pointers.

**Prototype**:

```
int find_max(int *arr, int size);
```

**Example**:

```
int arr[5] = {3, 9, 2, 8, 5};
int max = find_max(arr, 5); // max = 9
```

---

### 4. `reverse_array`

**Description**: Reverse the contents of an integer array in-place using pointers.

**Prototype**:

```
void reverse_array(int *arr, int size);
```

**Example**:

```
int arr[5] = {1, 2, 3, 4, 5};
reverse_array(arr, 5); // arr = {5, 4, 3, 2, 1}
```

---

### 5. `strlen_ptr`

**Description**: Implement a function that calculates the length of a string (null-terminated character array) using pointers.

**Prototype**:

```
int strlen_ptr(char *str);
```

**Example**:

```
char str[] = "Hello";
int length = strlen_ptr(str); // length = 5
```

---

### 6. `count_vowels`

**Description**: Count the number of vowels in a string using pointers.

**Prototype**:

```
int count_vowels(char *str);
```

**Example**:

```
char str[] = "OpenAI";
int vowels = count_vowels(str); // vowels = 3 ('O', 'e', 'A')
```

---

### 7. `copy_string`

**Description**: Copy the contents of one string to another using pointers (you may assume the destination has enough space).

**Prototype**:

```
void copy_string(char *dest, char *src);
```

**Example**:

```
char src[] = "Hello";
char dest[10];
copy_string(dest, src); // dest = "Hello"
```

---

### 8. `concat_strings`

**Description**: Concatenate two strings using pointers (you may assume the destination has enough space).

**Prototype**:

```
void concat_strings(char *dest, char *src);
```

**Example**:

```
char dest[20] = "Hello, ";
char src[] = "World!";
concat_strings(dest, src); // dest = "Hello, World!"
```

---

### 9. `sum_array`

**Description**: Compute the sum of all elements in an integer array using pointers.

**Prototype**:

```
int sum_array(int *arr, int size);
```

**Example**:

```
int arr[4] = {1, 2, 3, 4};
int sum = sum_array(arr, 4); // sum = 10
```

---

### 10. `compare_strings`

**Description**: Compare two strings lexicographically using pointers. Return `0` if they are equal, a negative value if the first string is lexicographically smaller, and a positive value if the first string is larger.

**Prototype**:

```
int compare_strings(char *str1, char *str2);
```

**Example**:

```
char str1[] = "apple";
char str2[] = "banana";
int cmp = compare_strings(str1, str2); // cmp < 0 since "apple" < "banana"
```

# Sample Main Function

Here's a sample `main` function to test some of the above functions:

```
#include <iostream>

int main() {
    // Test swap_ints
    int x = 5, y = 10;
    std::cout << "Before swap: x = " << x << ", y = " << y << std::endl;
    swap_ints(&x, &y);
    std::cout << "After swap: x = " << x << ", y = " << y << std::endl;

    // Test strlen_ptr
    char str[] = "Hello, world!";
    int length = strlen_ptr(str);
    std::cout << "Length of \"" << str << "\": " << length << std::endl;

    // Test reverse_array
    int arr[5] = {1, 2, 3, 4, 5};
    std::cout << "Original array: ";
    for(int i = 0; i < 5; i++) {
        std::cout << *(arr + i) << " ";
    }
    std::cout << std::endl;

    reverse_array(arr, 5);
    std::cout << "Reversed array: ";
    for(int i = 0; i < 5; i++) {
        std::cout << *(arr + i) << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

# Note:

- Correct implementation of all 10 functions.
- No use of array indexing (`arr[i]`) in any function—use only pointer arithmetic.
- Each function should perform the required task as described.
- Proper memory management and code readability.

# Part 2

1. Implement the linkedlist list code from the lecture notes.
2. Create a new classes called DoublyLinkedList and a DoublyLinkedNode. DoublyLinkedNode has both a prevPointer and nextPointer
3. Implement and **InsertAtBeginning , InsertAtEnd, and InsertAtPosition**